SafeRoute AI Intelligent Campus Evacuation Planner

Academic Project Report

Course Artificial Intelligence

Student name	Sugash K
Register number	24BCE2565

Submitted in partial fulfilment of the academic course requirements

Abstract

SafeRoute AI is an explainable campus evacuation planner that applies classical artificial intelligence to route selection in a changing emergency environment. The application represents a campus as a weighted graph and compares Breadth First Search, Depth First Search, Greedy Best First Search, and risk aware A star. Its principal result is that the physically shortest route is not always the best evacuation route: a longer alternative can have a lower weighted cost when hazard probability, congestion, or blocked corridors are considered.

The system provides five interactive modules: Live map, Compare, Replanning, Multi agent, and Map editor. The Live map displays the hazard value of every node, the weighted cost of each selected route edge, and a mathematical verification showing that the edge costs sum to the reported route cost. Replanning demonstrates how an agent updates its plan after a corridor becomes unavailable. Multi agent planning assigns several evacuees to exits subject to capacity and shared corridor penalties. Nine automated tests verify search correctness, hazard avoidance, blocked edge handling, replanning, map reconstruction, multi agent capacity, probability calculations, and visual cost labels.

The implementation is an educational prototype rather than a certified life safety system. It demonstrates state space search, informed search, utility based decision making, planning in a dynamic environment, and multi agent coordination in a form that can be inspected and explained during assessment.

Keywords

Artificial intelligence, state space search, A star, graph search, evacuation planning, dynamic replanning, multi agent planning, hazard aware routing, explainable AI

Report structure

- Problem definition, objectives, scope, and course concept mapping
- System architecture, data representation, and mathematical cost model
- Search algorithms and functional modules
- Experimental results, automated testing, limitations, and future work
- Appendices containing the project structure and local execution procedure

1 Introduction

1 1 Background

Evacuation route selection is commonly described as a shortest path problem, but emergency movement involves more than distance. Smoke, chemical leaks, crowd density, and corridor closures can make a short path unsafe or unusable. A useful planner must represent the environment, compare alternative actions, incorporate changing costs, and produce a new plan when its assumptions become invalid.

1 2 Problem statement

The project addresses the following question: how can a goal based agent choose and explain a campus evacuation route when distance, hazard probability, congestion, and unexpected corridor closures affect the quality of each action?

1 3 Objectives

1. Represent campus locations and corridors as a searchable weighted graph.
2. Implement BFS, DFS, Greedy Best First Search, and A star directly in Python.
3. Combine distance, average node hazard, and corridor congestion in one transparent cost function.
4. Show route edge costs and node hazard values directly on the Live map.
5. Demonstrate replanning after an unexpected environmental change.
6. Coordinate multiple agents while respecting exit capacity and route conflicts.
7. Allow a user to edit and validate a custom campus graph.
8. Verify important behaviour with reproducible experiments and automated tests.

1 4 Scope

The scope is limited to a small simulated campus graph with manually defined hazard and congestion values. The planner calculates recommendations but does not connect to physical sensors, control doors, or issue real emergency instructions. The interface is designed for academic demonstration and algorithm comparison.

2 Course Concept Mapping

Course concept	Project implementation	Evidence in application
State space representation	CampusGraph stores nodes, edges, hazards, congestion, and blocked corridors.	Live map and Map editor
Uninformed search	BFS uses a queue and DFS uses a stack.	Compare page
Informed search	Greedy uses h(n); A star uses g(n) + h(n).	Search strategy control and Compare page
Intelligent agents	A goal based, utility sensitive agent selects an exit route.	Route result and weighted cost
Planning	The plan is recomputed from the current location after a corridor closes.	Replanning page
Multi agent planning	Joint exit assignment includes capacity and shared corridor penalties.	Multi agent page
Knowledge representation	Hazard probabilities and corridor properties encode environmental state.	Scenario presets and Map editor
Evaluation	Algorithms are compared by cost, distance, risk, congestion, expanded nodes, and runtime.	Compare page and experiment output

2 1 Agent classification

SafeRoute AI is a goal based and utility sensitive agent. Reaching the selected exit is the goal; minimizing the weighted route cost is the utility objective. The environment is sequential because one movement changes the next state, dynamic because conditions can change during execution, partially observable because future changes are not known in advance, and nondeterministic because a corridor may unexpectedly become unavailable.

3 System Design

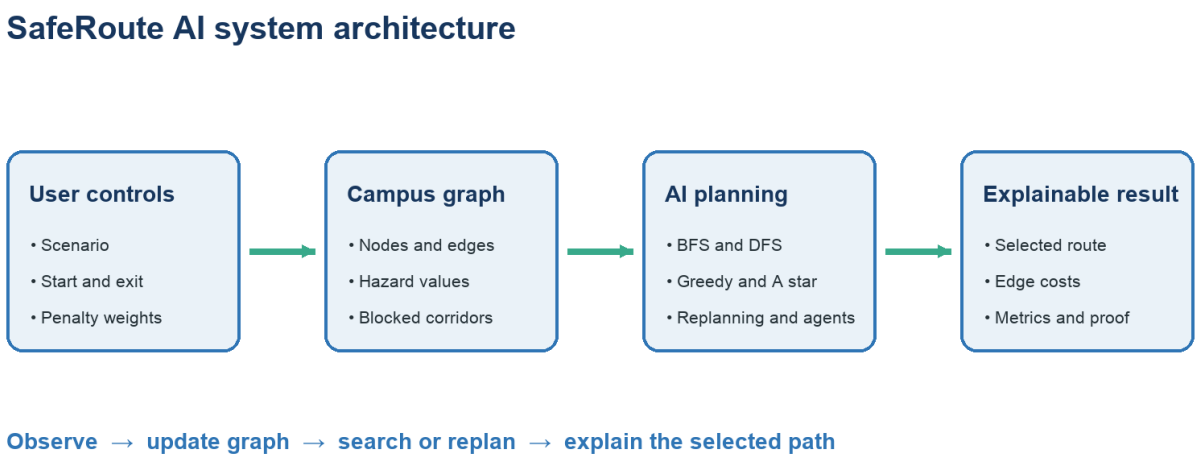


Figure 1 SafeRoute AI processing architecture

3 1 Input layer

The Streamlit interface collects the emergency scenario, starting location, target exit, search strategy, hazard penalty, and congestion penalty. A user can also activate a validated custom map.

3 2 Environment model

CampusGraph stores coordinates for drawing and heuristic calculation, adjacency lists for connectivity, hazard probabilities for nodes, congestion values for edges, and a set of blocked edges. The neighbours function filters blocked corridors before any search algorithm can expand them.

3 3 Planning layer

The selected algorithm searches the graph and records parent links, expansion order, accumulated cost, and runtime. A star and the multi agent planner use the weighted cost function during decision making. The replanning module modifies a copy of the graph and searches again from the current location.

3 4 Explanation layer

Plotly renders the graph, selected path, blocked corridors, node hazards, and route edge costs. Tables and metrics explain the final result. The mathematical proof below the Live map substitutes the current values into the same cost function used by the search implementation.

4 Mathematical Model

4 1 Graph formulation

The campus is an undirected weighted graph G = (V, E). Each vertex v in V is a campus location. Each edge (u, v) in E is a corridor that can be traversed in both directions unless it is blocked. A search state is the current vertex. An action moves to an unblocked neighbouring vertex, and the goal test checks whether the current vertex is the selected exit.

4 2 Edge cost

$$\text{mean_hazard}(u,v) = [H(u) + H(v)] / 2$$

$$C(u,v) = d(u,v)\,[1 + w_h\,\text{mean_hazard}(u,v) + w_c\,Q(u,v)]$$

Here d is corridor distance, H is node hazard probability, Q is corridor congestion, w_h is the user selected hazard penalty, and w_c is the congestion penalty. All terms are nonnegative, so the weighted cost is never lower than the physical distance.

4 3 Default route proof

Selected edge	Substitution	Cost
Library to Computer Lab	5.00 [1 + 5.0(0.00) + 2.0(0.25)]	7.50
Computer Lab to Science Lab	4.00 [1 + 5.0(0.00) + 2.0(0.15)]	5.20
Science Lab to South Exit	4.00 [1 + 5.0(0.00) + 2.0(0.10)]	4.80

$$\text{Route cost} = 7.50 + 5.20 + 4.80 = 17.50$$

The Live map calculates these values from the active graph rather than displaying fixed labels. Changing the scenario or either penalty causes Streamlit to rerun the search, update the path, and regenerate the proof.

4 4 A star evaluation function

$$f(n) = g(n) + h(n)$$

The value g(n) is the best known accumulated weighted cost from the start to n. The heuristic h(n) is straight line distance from n to the exit. Since weighted edge cost is at least physical distance, the straight line estimate remains a lower bound when edge distances are consistent with the map. Under this assumption the heuristic is admissible.

5 Search Algorithms

Algorithm	Frontier	Selection rule	Main limitation
BFS	FIFO queue	Shallowest unvisited state	Minimizes edges, not weighted safety cost
DFS	LIFO stack	Deepest available branch	Result depends on neighbour order
Greedy	Priority queue	Lowest h(n)	Ignores accumulated cost
A star	Priority queue	Lowest g(n) + h(n)	Quality depends on graph and heuristic assumptions

5 1 Breadth First Search

BFS expands the shallowest state first and is complete on this finite graph. With a visited set and adjacency list its graph search complexity is O(V + E). It is optimal only for the fewest edge objective when all actions have equal cost.

5 2 Depth First Search

DFS follows one branch before backtracking. The iterative implementation uses a Python list as a stack. It can reach the exit with a small frontier, but the first solution need not be short, safe, or low cost.

5 3 Greedy Best First Search

Greedy search prioritizes only the estimated remaining distance. It often expands few nodes but can enter a route with high accumulated risk because it does not include g(n) in its priority.

5 4 Risk aware A star

A star maintains a g score for each discovered node, updates a parent map when a cheaper path is found, and orders its heap by g(n) + h(n). This is the primary risk aware strategy because its accumulated cost includes distance, mean hazard, and congestion.

6 Functional Modules

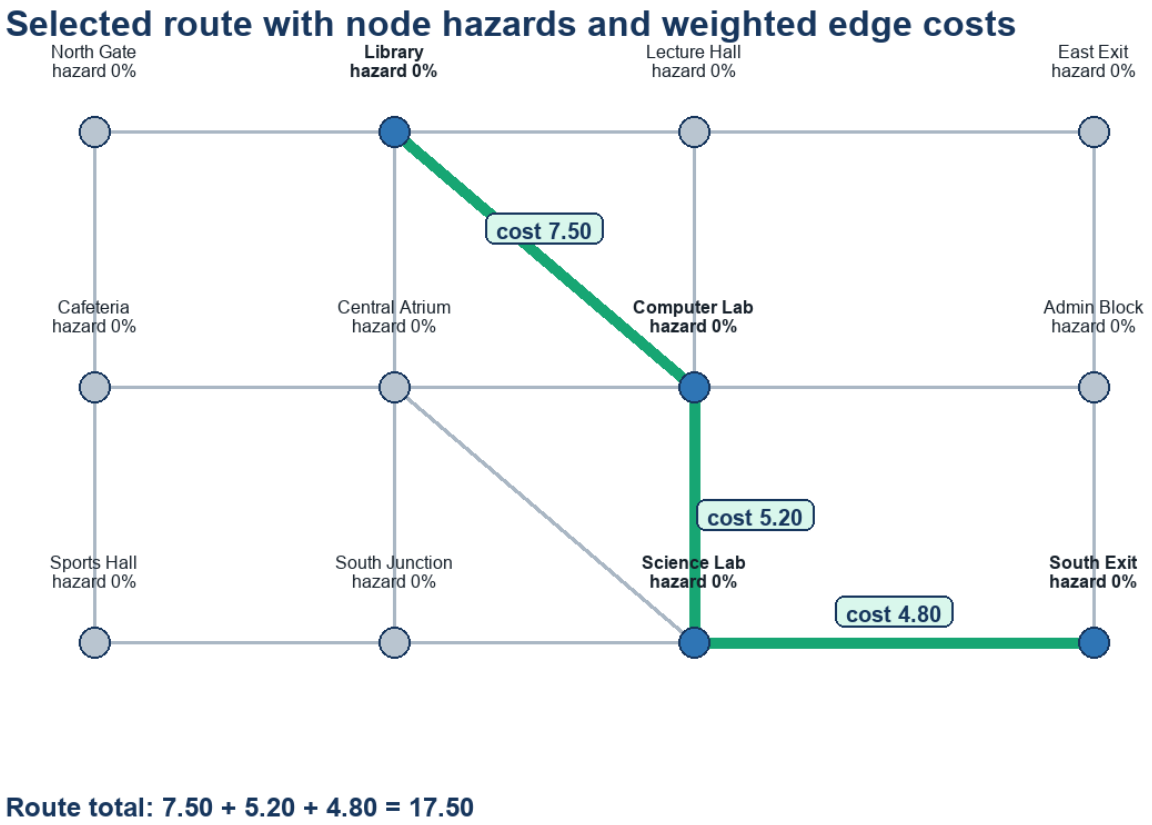


Figure 2 Live map with hazard labels and selected edge costs

6 1 Live map

The Live map shows the selected route as a mint line. Every node label displays its hazard probability, and each selected edge has a direct weighted cost label. Hover information provides distance, congestion, average hazard, and weighted cost. The metrics below the map report total weighted cost, physical distance, nodes expanded, and runtime.

6 2 Compare

The Compare page runs all four algorithms on the same graph and presents reached status, weighted cost, distance, risk exposure, congestion exposure, expanded nodes, runtime, and route. This separates route quality from search effort and prevents a single metric from being treated as universally decisive.

6 3 Map editor

The Map editor exposes node and corridor tables. Validation rejects empty or duplicate node names, missing edge endpoints, nonpositive distances, and probability values outside zero to one. The custom graph remains separate from the reproducible scenario presets until the user activates it.

7 Dynamic Replanning and Multi Agent Planning

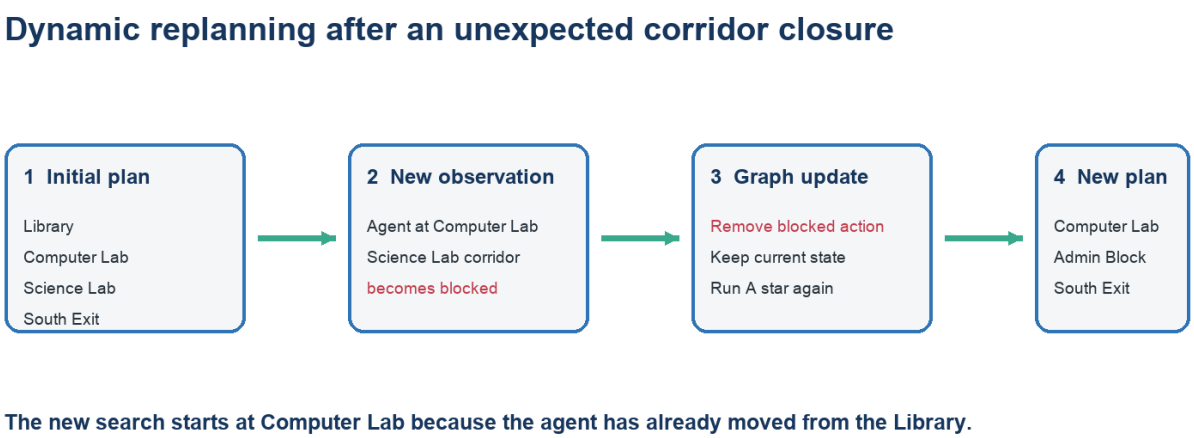


Figure 3 Observe update and replan sequence

7 1 Replanning procedure

1. Calculate the initial route from the selected start to the exit.
2. Simulate movement to the next location on that route.
3. Observe that a corridor in the remaining plan has become blocked.
4. Add the corridor to the blocked edge set.
5. Discard the invalid remainder and run A star from the current location.
6. Display the revised route or report that no alternative remains.

7 2 Multi agent objective

$$J = \text{sum}(\text{route_cost_i}) + \text{conflict_penalty} \times \text{shared_corridor_uses}$$

The centralized planner evaluates feasible assignments of selected agents to available exits. It enforces a maximum number of agents per exit and increases the joint objective when routes reuse the same corridor. For the default three agent experiment, the objective is 13.60 with zero shared corridors.

Agent	Start	Assigned exit	Cost	Route
Agent 1	Library	North Gate	3.60	Library to North Gate
Agent 2	Cafeteria	North Gate	5.20	Cafeteria to North Gate
Agent 3	Science Lab	South Exit	4.80	Science Lab to South Exit

8 Implementation

8 1 Technology stack

Technology	Purpose
Python	Core implementation language
Streamlit	Interactive local web interface
Plotly	Campus graph and route visualization
Pandas	Algorithm comparison and editable tables
Pytest	Automated correctness verification

8 2 Important data structures

- Adjacency dictionary for corridor connectivity
- Deque for the BFS frontier
- Python list for the DFS stack
- Heap based priority queue for Greedy and A star
- Parent dictionary for route reconstruction
- g score dictionary for the cheapest accumulated A star cost
- Set of unordered node pairs for blocked corridors

8 3 Explainability

The project intentionally avoids using NetworkX to calculate routes. Direct implementations make the queue, stack, heap, parent map, heuristic, and cost updates visible in the source code. The Live map and proof section connect those internal calculations to values that can be inspected during presentation.

8 4 Project organization

The user interface is contained in app.py. The evacuation package separates models, algorithms, scenarios, planning, multi agent logic, risk utilities, and visualization. Scripts generate reproducible experiment results, while the tests directory contains automated verification.

9 Results and Discussion

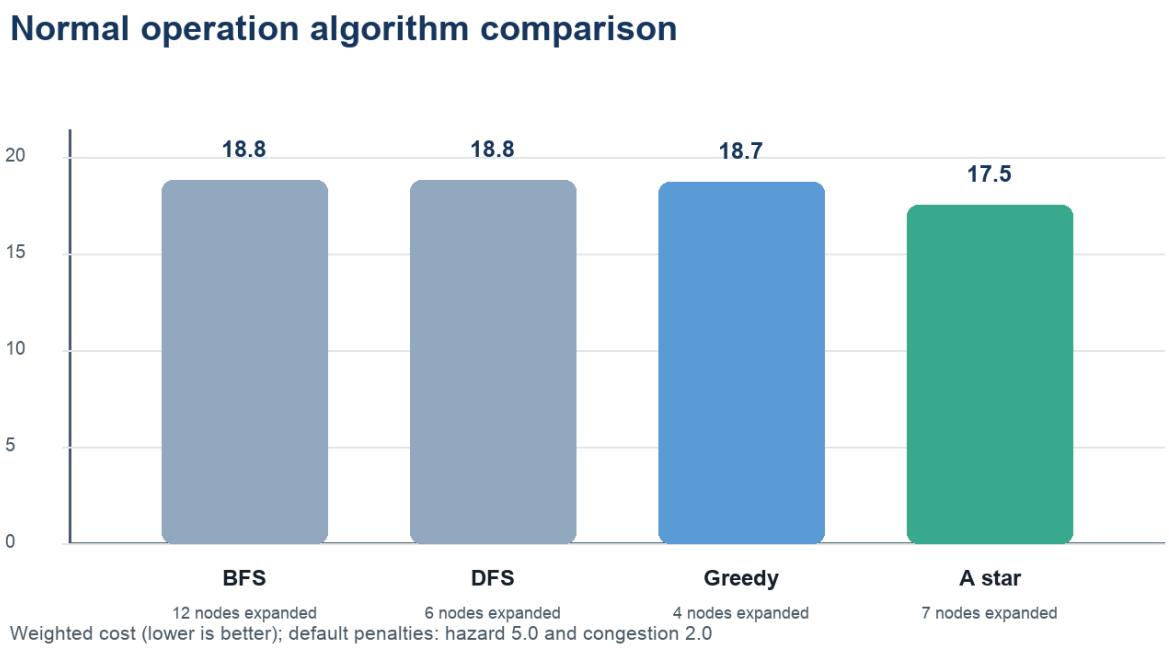


Figure 4 Default scenario weighted cost and search effort

Algorithm	Weighted cost	Distance	Nodes expanded	Selected route summary
BFS	18.80	13.0	12	Via Central Atrium and Science Lab
DFS	18.80	13.0	6	Via Central Atrium and Science Lab
Greedy	18.70	13.0	4	Via Computer Lab and Admin Block
A star	17.50	13.0	7	Via Computer Lab and Science Lab

Under normal operation, all reported routes have the same physical distance, but their congestion exposure differs. A star returns the lowest weighted cost of 17.50. Greedy expands fewer nodes, which demonstrates that search effort and route quality are different evaluation dimensions.

9 1 Hazard scenario evidence

During the Science lab leak scenario, the Science Lab hazard is 0.98, South Junction is 0.45, and Computer Lab is 0.30. With the standard penalties, BFS and DFS continue through the hazardous southern region and return a weighted cost of 40.85. A star instead selects Library to Lecture Hall to East Exit to Admin Block to South Exit, a longer 15.0 distance route with a lower weighted cost of 19.10 and zero modeled risk exposure.

9 2 Interpretation

The results support the project objective: minimizing geometric distance or expanded nodes alone does not guarantee the best safety aware route. The weighted objective allows the agent to trade additional distance for reduced hazard and congestion while remaining explainable through per edge calculations.

10 Testing and Verification

The final implementation passes nine automated tests. The tests are deterministic and run locally with Pytest.

Test area	Expected behaviour
All search algorithms	Every algorithm reaches the exit on the base graph.
Severe Library fire	A star avoids the high hazard Library when a safer alternative exists.
Blocked corridors	Returned paths never traverse an edge marked as blocked.
Probability utility	Positive evidence increases the calculated hazard probability.
No path case	The search returns reached false and an empty path.
Dynamic replanning	The revised plan excludes the newly blocked corridor.
Multi agent capacity	No exit receives more agents than its configured capacity.
Editable map	A graph reconstructed from editor tables preserves connectivity.
Visual calculation labels	Displayed edge costs sum to the route cost and node hazards are shown.

10 1 Verification results

- Pytest result: 9 passed
- Default selected edge labels: 7.50, 5.20, and 4.80
- Verified sum: 17.50, equal to the reported weighted route cost
- Streamlit health endpoint returns a successful response while the application is running
- The packaged project archive passes integrity verification

11 Limitations Ethics and Future Work

11 1 Limitations

- The campus graph and scenario values are simulated rather than measured by certified sensors.
- Hazard values do not automatically spread over time according to a physical fire or gas model.
- The multi agent assignment method enumerates combinations and is suitable only for a small classroom graph.
- The model does not yet include mobility restrictions, stairs, smoke visibility, travel speed, or human behaviour.
- Runtime measurements on a small graph are illustrative and should not be treated as large scale benchmarks.

11 2 Ethical and safety considerations

SafeRoute AI is not a certified life safety product. A real deployment would require validated building maps, calibrated and redundant sensors, accessibility planning, secure communications, privacy controls, failure recovery, human oversight, regulatory approval, and alignment with official emergency procedures. The application must not be used as the sole basis for real evacuation decisions.

11 3 Future work

- Integrate real time sensor and crowd density data.
- Model time dependent hazard spread and corridor capacity.
- Add accessible routing for users with different mobility needs.
- Replace exhaustive multi agent assignment with a scalable coordination method.
- Provide mobile guidance and offline fallback behaviour.
- Evaluate the planner on larger validated building maps.

12 Conclusion

SafeRoute AI demonstrates how classical artificial intelligence can convert a campus evacuation problem into an explainable state space search task. The project implements four search strategies, a transparent weighted cost model, dynamic replanning, multi agent coordination, editable graph validation, and visible mathematical verification. Experimental results show that A star can choose a safer or less congested route even when another algorithm expands fewer nodes or follows an equally short physical path. The project therefore meets its academic objective while clearly separating an educational simulation from a real safety certified system.

References

1. Russell, S. J., and Norvig, P. Artificial Intelligence A Modern Approach. Fourth edition, Pearson, 2021.
2. Hart, P. E., Nilsson, N. J., and Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 1968.
3. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
4. Streamlit. Streamlit Documentation. https://docs.streamlit.io/
5. Plotly. Plotly Python Graphing Library Documentation. https://plotly.com/python/
6. Pandas Development Team. Pandas Documentation. https://pandas.pydata.org/docs/
7. Pytest Development Team. Pytest Documentation. https://docs.pytest.org/

Appendix A Project Structure

File or folder	Responsibility
app.py	Streamlit interface and page coordination
evacuation/models.py	Graph, edge, and result data models
evacuation/algorithms.py	BFS, DFS, Greedy, and A star
evacuation/scenarios.py	Campus map and emergency scenarios
evacuation/planning.py	Unexpected blockage replanning
evacuation/multiagent.py	Exit assignment and conflict penalties
evacuation/visualization.py	Plotly map and direct calculation labels
tests/	Automated correctness tests
scripts/run_experiments.py	Reproducible experiment generator

Appendix B Local Execution

1. Open a terminal in the project folder.
2. Create a virtual environment with python3 -m venv .venv.
3. Activate it with source .venv/bin/activate on macOS or Linux.
4. Install dependencies with python -m pip install -r requirements.txt.
5. Run the website with streamlit run app.py.
6. Open http://localhost:8501 in a browser.
7. Run tests with pytest -q before submission.